

Guia Técnico DevOps: Correções Estruturadas & Branching

Manual de execução passo a passo para resolução de falhas nos testes do FastAPI

Objetivo: Este documento orienta o desenvolvedor na criação de ramificações (branches) isoladas e na aplicação das correções de código necessárias para sanar as falhas identificadas pelo *pytest*.

Tarefa 1: Validação do Formulário de Contato

Erro identificado: `test_empty_contact_returns_400 - assert 201 == 400`

Motivo: O sistema aceita mensagens vazias e retorna status 201. Deve validar e retornar 400 se faltar informação.

Fluxo de Execução no Terminal:

```
git checkout main
git checkout -b fix/contact-validation
```

Código a ser modificado em `app/routers/contact.py`:

```
from fastapi import APIRouter, HTTPException, status
from pydantic import BaseModel, Field

router = APIRouter(prefix="/contact", tags=["Contact"])

class ContactSchema(BaseModel):
    name: str = Field(..., min_length=1)
    email: str = Field(..., min_length=1)
    message: str = Field(..., min_length=1)

@router.post("")
def send_contact(contact: ContactSchema):
    if not contact.name.strip() or not contact.email.strip() or not contact.message.strip():
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Campos obrigatorios nao podem estar vazios"
        )
    return {"message": "Contato enviado com sucesso!"}
```

Comandos de Finalização e Envio:

```
python -m pytest
git add app/routers/contact.py
git commit -m "fix(contact): adiciona validacao de erro 400 para campos vazios"
git push -u origin fix/contact-validation
```

Tarefa 2: Correção dos Status Codes de Cursos

Erros identificados: Listagem retornando 201 em vez de 200; Busca inexistente retornando 200 em vez de 404; Exclusão retornando 200 em vez de 204.

Fluxo de Execução no Terminal:

```
git checkout main
git checkout -b fix/courses-status-codes
```

Código a ser modificado em `app/routers/courses.py`:

```
from fastapi import APIRouter, HTTPException, status

router = APIRouter(prefix="/courses", tags=["Courses"])

@router.get("", status_code=status.HTTP_200_OK)
def list_courses(active: bool = True):
    # Correcao: Garantir status_code=200 na listagem
    return courses_list

@router.get("/{course_id}", status_code=status.HTTP_200_OK)
def get_course(course_id: int):
    course = courses_service.find_by_id(course_id)
    if not course:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Curso nao encontrado"
        )
    return course

@router.delete("/{course_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_course(course_id: int):
    # Correcao: Retornar 204 No Content sem corpo na resposta
    return None
```

Comandos de Finalização e Envio:

```
python -m pytest
git add app/routers/courses.py
git commit -m "fix(courses): corrige status codes HTTP de listagem, busca e exclusao"
git push -u origin fix/courses-status-codes
```